

13 강

머신러닝

귀널 기법 3

통계·데이터과학과 박준희



✦ 학습목차

- 1 One-Class Problem
- 2 Support Vector Data Description
- 3 One-Class ν -SVM
- 4 SVM 파이썬 실습



1

One-Class Problem



1 One-Class Problem

● One-Class Problem 이란?

- **Goal:** 한 클래스(정상이라 가정)를 다른 모든 객체(비정상)로부터 구분하고 싶음
- **Training 데이터:** 정상 클래스에 대한 샘플 $\{\mathbf{x}_i\}_{i=1}^N$ 만이 주어짐
 - 비정상 클래스(Outlier) 샘플은 거의 없거나 전혀 없다고 가정
- **아이디어:** 주어진 데이터가 형성하는 **주요 지지(support)** 영역을 찾아서, 그 영역 밖의 샘플을 "비정상"으로 판정

2

Support Vector Data Description



2 Support Vector Data Description

● Support Vector Data Description (SVDD)

SVDD는 서포트 벡터 머신의 아이디어를 활용하여, 한 클래스를 감싸는 최소 구(球)(혹은 더 일반적으로 최소 볼륨의 형태)로 정의한다.

1. 목표: 입력 데이터 $\{\mathbf{x}_i \in \mathbb{R}^d\}$ 를 모두 포함(혹은 대부분 포함)하는 반지름 R 을 최소화하는 구를 찾는다.
 - 중심: $\mathbf{a} \in \mathbb{R}^d$
 - 반지름: R (스칼라)
2. 이후 어떤 새로운 점 $\mathbf{x}$ 가 이 구 밖에 있으면 (즉, $\|\mathbf{x} - \mathbf{a}\| > R$) 비정상 (outlier)으로 본다.

2 Support Vector Data Description

● Hard-Margin SVDD

- 결정 함수 (Decision Function)

$$f(\mathbf{x}) = \|\mathbf{x} - \mathbf{a}\|^2 - R^2$$

- $\mathbf{a}$: 구의 중심.
- R : 구의 반지름.
- $f(\mathbf{x}) \leq 0$: $\mathbf{x}$ 가 구의 경계 안에 있거나 경계 위에 있음 -> 정상(normal)
- $f(\mathbf{x}) > 0$: $\mathbf{x}$ 가 구의 경계 밖에 있음 -> 이상치(outlier)

2 Support Vector Data Description

● Hard-Margin SVDD

■ Primal Formulation

$$\begin{aligned} \min_{R^2, \mathbf{a}} \quad & R^2 \\ \text{subject to} \quad & \|\mathbf{x}_i - \mathbf{a}\|^2 \leq R^2, \quad i = 1, \dots, N. \end{aligned}$$

- 즉, $\|\mathbf{x}_i - \mathbf{a}\|$ 가 R 이하가 되도록 하는 최소 R

(커널화 시 $\|\phi(\mathbf{x}_i) - \phi(\mathbf{a})\| \leq R$)

2 Support Vector Data Description

● Hard-Margin SVDD

- Lagrangian

라그랑주 승수 $\alpha_i \geq 0$ 를 붙여서 듀얼 문제를 정리하면:

$$\begin{aligned}\mathcal{L}(R^2, \mathbf{a}, \boldsymbol{\alpha}) &= R^2 + \sum_{i=1}^N \alpha_i (\|\mathbf{x}_i - \mathbf{a}\|^2 - R^2) \\ &= R^2 \left(1 - \sum_{i=1}^N \alpha_i \right) + \sum_{i=1}^N \alpha_i \|\mathbf{x}_i - \mathbf{a}\|^2.\end{aligned}$$

2 Support Vector Data Description

● Hard-Margin SVDD

- Stationarity 조건

$$(1) \frac{\partial \mathcal{L}}{\partial R^2} = 0$$

$$\sum_{i=1}^N \alpha_i = 1$$

$$(2) \frac{\partial \mathcal{L}}{\partial \mathbf{a}} = 0$$

$$\mathbf{a} = \sum_{i=1}^N \alpha_i \mathbf{x}_i.$$

2 Support Vector Data Description

● Hard-Margin SVDD

- Dual 최적화

$$\begin{aligned} \max_{\alpha} \quad & \sum_{i=1}^N \alpha_i (\mathbf{x}_i \cdot \mathbf{x}_i) - \sum_{i=1}^N \sum_{j=1}^N \alpha_i \alpha_j (\mathbf{x}_i \cdot \mathbf{x}_j) \\ \text{subject to} \quad & \sum_{i=1}^N \alpha_i = 1, \quad \alpha_i \geq 0, \quad i = 1, \dots, N. \end{aligned}$$

2 Support Vector Data Description

● Soft-Margin SVDD

- 실제로 모든 점을 구 안에 완벽히 포함하기 어렵거나 오버피팅이 우려될 수 있으므로, 슬랙 변수 $\xi_i \geq 0$ 도입
- 제약: $\|\mathbf{x}_i - \mathbf{a}\|^2 \leq R^2 + \xi_i$
- 목적함수: $\min R^2 + C \sum_i \xi_i$
- 듀얼 문제에서는 $0 \leq \alpha_i \leq C$ 제약이 들어감 (C-SVM과 유사).
- 최적해 α_i 로부터 중심 $\mathbf{a}$ 와 반지름 R 을 구하고, 이상치(Outlier) 혹은 경계에서 벗어난 점들에 대해서는 ξ_i 가 양수가 되어 패널티를 부과한다.

2 Support Vector Data Description

● Soft-Margin SVDD

■ Primal 문제

$$\begin{aligned} \min_{R^2, \mathbf{a}, \xi} \quad & R^2 + C \sum_{i=1}^N \xi_i \\ \text{subject to} \quad & \|\mathbf{x}_i - \mathbf{a}\|^2 \leq R^2 + \xi_i, \quad i = 1, \dots, N, \\ & \xi_i \geq 0, \quad i = 1, \dots, N. \end{aligned}$$

- $R \geq 0$: 구의 반지름
- $\mathbf{a} \in \mathbb{R}^d$: 구의 중심
- $\xi_i \geq 0$: 슬랙 변수 (i 번째 점이 경계를 벗어난 정도)
- $C > 0$: 슬랙 변수에 대한 패널티를 조절하는 하이퍼파라미터

2 Support Vector Data Description

● Soft-Margin SVDD

- Lagrangian (듀얼 문제) 도출

Primal 문제의 제약식을 반영하기 위해 **라그랑주 승수** 도입:

- $\alpha_i \geq 0 : \|\mathbf{x}_i - \mathbf{a}\|^2 \leq R^2 + \xi_i$ 제약에 대한 승수
- $\gamma_i \geq 0 : \xi_i \geq 0$ 제약에 대한 승수

$$\mathcal{L}(R^2, \mathbf{a}, \boldsymbol{\xi}, \boldsymbol{\alpha}, \boldsymbol{\gamma}) = R^2 + C \sum_{i=1}^N \xi_i - \sum_{i=1}^N \alpha_i (R^2 + \xi_i - \|\mathbf{x}_i - \mathbf{a}\|^2) - \sum_{i=1}^N \gamma_i \xi_i.$$

2 Support Vector Data Description

● Soft-Margin SVDD

- KKT 조건 (Stationarity 조건)

$$(1) \frac{\partial \mathcal{L}}{\partial R^2} = 0$$

$$1 - \sum_{i=1}^N \alpha_i = 0 \quad \Rightarrow \quad \sum_{i=1}^N \alpha_i = 1.$$

2 Support Vector Data Description

● Soft-Margin SVDD

- KKT 조건 (Stationarity 조건)

$$(2) \frac{\partial \mathcal{L}}{\partial \mathbf{a}} = 0$$

$$\frac{\partial}{\partial \mathbf{a}} \left(- \sum_i \alpha_i \cdot (-\|\mathbf{x}_i - \mathbf{a}\|^2) \right) = \frac{\partial}{\partial \mathbf{a}} \left(\sum_i \alpha_i \|\mathbf{x}_i - \mathbf{a}\|^2 \right) = 0.$$

$$\mathbf{a} = \sum_{i=1}^N \alpha_i \mathbf{x}_i.$$

2 Support Vector Data Description

● Soft-Margin SVDD

- KKT 조건 (Stationarity 조건)

$$(3) \frac{\partial \mathcal{L}}{\partial \xi_i} = 0$$

$$C - \alpha_i - \gamma_i = 0 \quad \implies \quad \alpha_i + \gamma_i = C.$$

즉, $\alpha_i \leq C$ 를 의미하고, 만약 $\alpha_i < C$ 이면 $\gamma_i > 0$ (슬랙 변수 제약을 조절하는 역할).

2 Support Vector Data Description

● Soft-Margin SVDD

- Dual 문제

$$\begin{aligned} \max_{\alpha} \quad & \sum_{i=1}^N \alpha_i (\mathbf{x}_i \cdot \mathbf{x}_i) - \sum_{i=1}^N \sum_{j=1}^N \alpha_i \alpha_j (\mathbf{x}_i \cdot \mathbf{x}_j) \\ \text{subject to} \quad & \sum_{i=1}^N \alpha_i = 1, \quad 0 \leq \alpha_i \leq C, \quad i = 1, \dots, N. \end{aligned}$$

2 Support Vector Data Description

● Soft-Margin SVDD

- 최종 결정 함수

$$f(\mathbf{x}) = \|\mathbf{x} - \mathbf{a}\|^2 - R^2.$$

- $\mathbf{a}$: 구의 중심. 학습 과정에서 $\mathbf{a} = \sum_{i=1}^N \alpha_i \mathbf{x}_i$ 으로 최적화됨.
- R^2 : 구의 반지름의 제곱. 학습 과정에서 $\|\mathbf{x}_s - \mathbf{a}\|^2$ ($\mathbf{x}_s$ 는 서포트 벡터) 으로 최적화됨.

$$\|\mathbf{x}_s - \mathbf{a}\|^2 = \mathbf{x}_s \cdot \mathbf{x}_s - 2 \sum_{i=1}^N \alpha_i (\mathbf{x}_s \cdot \mathbf{x}_i) + \sum_{i=1}^N \sum_{j=1}^N \alpha_i \alpha_j (\mathbf{x}_i \cdot \mathbf{x}_j).$$

2 Support Vector Data Description

● Soft-Margin SVDD

■ 커널 확장

$\mathbf{x}_i$ 대신 $\phi(\mathbf{x}_i)$ 를 사용하고, 내적 $\phi(\mathbf{x}_i) \cdot \phi(\mathbf{x}_i)$ 대신 $k(\mathbf{x}_i, \mathbf{x}_j)$ 를 사용하는 커널 트릭을 적용하면,

$$\|\phi(\mathbf{x}_s) - \phi(\mathbf{a})\|^2 = k(\mathbf{x}_s, \mathbf{x}_s) - 2 \sum_i \alpha_i k(\mathbf{x}_s, \mathbf{x}_i) + \sum_{i,j} \alpha_i \alpha_j k(\mathbf{x}_i, \mathbf{x}_j).$$

$\alpha_s > 0$ 인 경계점 $\mathbf{x}_s$, 즉, support vector인 경우에 대해 이 결과가 곧 R^2 이 된다.

그리하여, 커널화된 SVDD의 최종 결정 함수는

$$f(\mathbf{x}) = k(\mathbf{x}, \mathbf{x}) - 2 \sum_i \alpha_i k(\mathbf{x}, \mathbf{x}_i) + \sum_{i,j} \alpha_i \alpha_j k(\mathbf{x}_i, \mathbf{x}_j) - R^2.$$

3

One-Class ν -SVM



3 One-Class ν -SVM

● One-Class ν -SVM (이후 One-Class SVM으로 표기)

- 목적: 훈련 데이터가 원점(Origin)에서 최대한 떨어지도록 하는 초평면을 찾음으로써, 고립된 점들을 Outlier로 간주.
- 개념:
 - 기존의 SVM은 데이터의 두 클래스를 분리하는 초평면을 찾는 데 초점을 둔다면,
 - **One-Class ν -SVM**은 "(유의미한) 분리 기준이 없어 보이는 단일 클래스" 데이터를 원점과 분리하는 초평면을 찾음.

3 One-Class v-SVM

● One-Class SVM

- 결정 함수 (Decision Function)

- 점 $\mathbf{x}$ 에 대하여

$$f(\mathbf{x}) = \text{sign}(\langle \mathbf{w}, \phi(\mathbf{x}) \rangle - \rho).$$

$f(\mathbf{x}) > 0$ 이면 정상 영역, $f(\mathbf{x}) < 0$ 이면 Outlier로 간주.

3 One-Class ν -SVM

● One-Class SVM

■ Primal Formulation

$$\min_{\mathbf{w}, \rho, \xi} \quad \frac{1}{2} \|\mathbf{w}\|^2 - \rho + \frac{1}{\nu N} \sum_{i=1}^N \xi_i,$$

subject to $\langle \mathbf{w}, \phi(\mathbf{x}_i) \rangle \geq \rho - \xi_i, \quad \xi_i \geq 0, \quad i = 1, \dots, N,$

- $\phi(\cdot)$ 는 커널 매핑
- $\mathbf{w}$: 고차원(커널 매핑 후)에서의 초평면 파라미터
- ρ : 초평면의 마진(또는 offset) 항
- ξ_i 는 슬랙 변수(Outlier 허용량)
- $\nu \in (0, 1]$ 는 이상치(Outlier) 비율 조절 하이퍼파라미터
- $\|\mathbf{w}\|^2$ 를 최소화하면서, ρ 를 최대화(즉, 원점과 데이터들의 간격을 최대화)

출처:파이썬 프로그램 실습내용 중

3 One-Class ν -SVM

● One-Class SVM

- Lagrangian (듀얼 문제) 도출

$$\begin{aligned}\mathcal{L}(\mathbf{w}, \rho, \boldsymbol{\xi}, \alpha, \beta) = & \frac{1}{2} \|\mathbf{w}\|^2 - \rho + \frac{1}{\nu N} \sum_{i=1}^N \xi_i \\ & - \sum_{i=1}^N \alpha_i \left(\langle \mathbf{w}, \phi(\mathbf{x}_i) \rangle - \rho + \xi_i \right) - \sum_{i=1}^N \beta_i \xi_i,\end{aligned}$$

여기서

$$\alpha_i \geq 0, \quad \beta_i \geq 0 \quad (i = 1, \dots, N).$$

3 One-Class ν -SVM

● One-Class SVM

- KKT 조건 - (1) Stationarity 조건

$$\frac{\partial \mathcal{L}}{\partial \mathbf{w}} = \mathbf{w} - \sum_{i=1}^N \alpha_i \phi(\mathbf{x}_i) = \mathbf{0}.$$

$$\mathbf{w} = \sum_{i=1}^N \alpha_i \phi(\mathbf{x}_i).$$

$$\frac{\partial \mathcal{L}}{\partial \rho} = -1 + \sum_{i=1}^N (\alpha_i) = 0.$$

$$\sum_{i=1}^N \alpha_i = 1.$$

$$\frac{\partial \mathcal{L}}{\partial \xi_i} = \frac{1}{\nu N} - \alpha_i - \beta_i = 0.$$

$$\alpha_i + \beta_i = \frac{1}{\nu N}.$$

3 One-Class v-SVM

● One-Class SVM

- KKT 조건 - (2) Primal Feasibility

$$\langle \mathbf{w}, \phi(\mathbf{x}_i) \rangle \geq \rho - \xi_i, \quad \xi_i \geq 0.$$

- (3) Dual Feasibility

$$\bullet \alpha_i \geq 0, \beta_i \geq 0$$

- (4) Complementary Slackness

$$\begin{aligned} \bullet \alpha_i (\langle \mathbf{w}, \phi(\mathbf{x}_i) \rangle - \rho + \xi_i) &= 0 \\ \bullet \beta_i \xi_i &= 0 \end{aligned}$$

3 One-Class ν -SVM

● One-Class SVM

- Dual 문제

$$\max_{\alpha} \quad -\frac{1}{2} \sum_{i=1}^N \sum_{j=1}^N \alpha_i \alpha_j k(\mathbf{x}_i, \mathbf{x}_j) \quad \left(\text{라그랑지안에서 } \|\mathbf{w}\|^2 \text{ 항에 해당} \right)$$

$$\text{subject to } \sum_{i=1}^N \alpha_i = 1,$$

$$0 \leq \alpha_i \leq \frac{1}{\nu N}, \quad i = 1, \dots, N.$$

3 One-Class ν -SVM

● One-Class SVM

- 최종 결정 함수

듀얼 해 $\{\alpha_i^*\}$ 를 구하면, Primal 변수 $\mathbf{w}$ 는

$$\mathbf{w}^* = \sum_{i=1}^N \alpha_i^* \phi(\mathbf{x}_i)$$

로 표현되며, 초평면의 기준(offset) ρ^* 는 KKT 조건에서 다음처럼 표현할 수 있다:

$$\rho^* = \langle \mathbf{w}^*, \phi(\mathbf{x}_s) \rangle = \sum_{j=1}^N \alpha_j^* k(\mathbf{x}_j, \mathbf{x}_s),$$

여기서 $\mathbf{x}_s$ 는 $0 < \alpha_s^* \leq \frac{1}{\nu N}$ 에 해당하는 샘플(= **Support Vectors**) 중 하나이다.

3 One-Class v-SVM

● One-Class SVM

- 최종 결정 함수

결정 함수는 새로운 점 $\mathbf{x}$ 에 대해 다음과 같이 정의된다:

$$f(\mathbf{x}) = \langle \mathbf{w}^*, \phi(\mathbf{x}) \rangle - \rho^* = \sum_{i=1}^N \alpha_i^* k(\mathbf{x}_i, \mathbf{x}) - \rho^*.$$

- $f(\mathbf{x}) > 0$ 이면 **정상 영역**(inlier)
- $f(\mathbf{x}) < 0$ 이면 **이상치 영역**(outlier)으로 판별

4

SVM 파이썬 실습



4 SVM 파이썬 실습

● 예제 데이터 생성

```
# "Normal" data in two clusters
N = 50
mean_1 = [-2, -2]
mean_2 = [2, 2]
cov = [[0.25, 0],
        [0, 0.25]]
X_1 = np.random.multivariate_normal(mean_1, cov, N//2)
X_2 = np.random.multivariate_normal(mean_2, cov, N//2)
X_normal = np.vstack([X_1, X_2])

# "Outlier" data
X_outliers = np.random.uniform(low=-6, high=6, size=(10, 2))

# Train data
X_train = np.vstack([X_normal, X_outliers])
```

출처:파이썬 프로그램 실습내용 중

4 SVM 파이썬 실습

● 공통 : RBF Kernel

```
def rbf_kernel(x, y, gamma=0.1):  
    diff = x - y  
    return np.exp(-gamma * np.sum(diff**2))  
  
def build_kernel_matrix(X, gamma=0.1):  
    N = len(X)  
    K = np.zeros((N, N))  
    for i in range(N):  
        for j in range(N):  
            K[i, j] = rbf_kernel(X[i], X[j], gamma=gamma)  
    return K
```

4 SVM 파이썬 실습

● SVDD : 듀얼 목적함수, 등식 제약 조건(공통)

```
def svdd_objective(alpha, K):  
    """  
    Minimizing:  $\sum_{i,j} \alpha_i \alpha_j K[i,j] - \sum_i \alpha_i K[i,i]$   
    Subject to:  $\sum_i \alpha_i = 1$   
                 $0 \leq \alpha_i \leq C$   
    """  
    return np.sum((alpha[:,None] * alpha[None,:]) * K) - np.sum(alpha * np.diag(K))  
  
def eq_constraint_sum_alpha(alpha):  
    #  $\sum_i \alpha_i = 1$   
    return np.sum(alpha) - 1.0
```

4 SVM 파이썬 실습

● SVDD : 학습 함수

```
def fit_svdd(X, C=1.0, gamma=0.1):  
    N = len(X)  
    K = build_kernel_matrix(X, gamma=gamma)  
  
    # initial alpha  
    alpha0 = np.ones(N) / N  
  
    # constraints: sum_i alpha_i = 1  
    cons = [{'type': 'eq', 'fun': eq_constraint_sum_alpha}]  
    # bounds: alpha_i in [0, C]  
    bnds = [(0, C)] * N
```

4 SVM 파이썬 실습

● SVDD : 학습 함수

```
res = minimize(  
    fun=svdd_objective,  
    x0=alpha0,  
    args=(K,),  
    constraints=cons,  
    bounds=bnds,  
    options={'disp': True}  
)  
alpha_opt = res.x
```

4 SVM 파이썬 실습

● SVDD : 학습 함수

```
sv_indices = np.where((alpha_opt > 1e-8) & (alpha_opt <= C))[0]
R2_list = []
for s in sv_indices:
    # sum_i alpha_i K[s,i]
    sum1 = np.sum(alpha_opt * K[s,:])
    # sum_{i,j} alpha_i alpha_j K[i,j]
    sum2 = np.sum((alpha_opt[:,None]*alpha_opt[None,:]) * K)
    R2_s = K[s,s] - 2*sum1 + sum2
    R2_list.append(R2_s)
R2 = np.mean(R2_list)
R = np.sqrt(max(R2, 0.0))
return alpha_opt, K, R
```

4 SVM 파이썬 실습

● SVDD : 결정 함수

```
def decision_function_svdd(x_new, alpha, X, K, R, gamma=0.1):
    N = len(X)
    # K(x_new, x_new)
    k_xx = rbf_kernel(x_new, x_new, gamma=gamma)
    # sum1 : sum_i alpha_i K(x_new, x_i)
    sum1 = 0.0
    for i in range(N):
        sum1 += alpha[i]*rbf_kernel(x_new, X[i], gamma=gamma)
    # sum2 : sum_{i,j} alpha_i alpha_j K(x_i, x_j)
    sum2 = np.sum((alpha[:,None]*alpha[None,:]) * K)
    dist2 = k_xx - 2*sum1 + sum2
    return dist2 - R**2 # negative: inside, inlier; positive: outside, outlier
```

4 SVM 파이썬 실습

● SVDD : 예측 함수

```
def predict_svdd(X_test):  
    pred = []  
    for x in X_test:  
        val = decision_function_svdd(x, alpha_svdd, X_normal, K_svdd, R_svdd, gamma=0.1)  
        if val <= 0:  
            pred.append(+1) # inside => normal  
        else:  
            pred.append(-1) # outlier  
    return np.array(pred)
```

4 SVM 파이썬 실습

● One-Class SVM : 듀얼 목적 함수

```
def oneclass_objective(alpha, K):
    # objective = 0.5 * sum_{i,j} alpha_i alpha_j K[i,j] (we'll minimize)
    return 0.5 * np.sum((alpha[:,None] * alpha[None,:]) * K)
```

$$\max_{\alpha} -\frac{1}{2} \sum_{i,j} \alpha_i \alpha_j K[i,j], \quad \sum_i \alpha_i = 1, \quad 0 \leq \alpha_i \leq \frac{1}{\nu N}$$

를 부호를 반대로 바꾸어서 `scipy.minimize` 에 맞춤.

4 SVM 파이썬 실습

● One-Class SVM : 학습 함수

```
def fit_oneclass_svm(X, nu=0.05, gamma=0.1):  
    N = len(X)  
    K = build_kernel_matrix(X, gamma=gamma)  
  
    # alpha in [0, 1/(nu*N)], sum_i alpha_i=1  
    alpha0 = np.ones(N)/N  
    cons = [{'type': 'eq', 'fun': eq_constraint_sum_alpha}]  
    upper = 1.0/(nu*N)  
    bnds = [(0, upper)]*N
```

4 SVM 파이썬 실습

● One-Class SVM : 학습 함수

```
res = minimize(  
    fun=oneclass_objective,  
    x0=alpha0,  
    args=(K,),  
    constraints=cons,  
    bounds=bnds,  
    options={'disp': True}  
)  
alpha_opt = res.x
```

4 SVM 파이썬 실습

● One-Class SVM : 학습 함수

```
# Then we get "rho" from KKT:
# pick s with  $0 < \alpha_s < 1/(nu*N)$ :
#  $\rho = \sum_j \alpha_j K[s,j]$ 
sv_indices = np.where((alpha_opt > 1e-8)&(alpha_opt < upper-1e-8))[0]
rho_list = []
for s in sv_indices:
    rho_s = np.sum(alpha_opt*K[s,:])
    rho_list.append(rho_s)
rho = np.mean(rho_list)
return alpha_opt, K, rho
```

4 SVM 파이썬 실습

● One-Class SVM : 결정 함수

```
def decision_function_oneclass(x_new, alpha, X, K, rho, gamma=0.1):
    #  $f(x) = \sum_i \alpha_i K(x_i, x) - \rho$ 
    val = 0.0
    for i in range(len(X)):
        val += alpha[i]*rbf_kernel(X[i], x_new, gamma=gamma)
    return -(val - rho) # positive: outlier, negative: normal로 바꿈
```

- $f(x) = \sum_i \alpha_i k(\mathbf{x}_i, \mathbf{x}) - \rho$
- One-Class SVM 결정함수의 본래 형태는 양수를 정상 데이터로, 음수를 이상치 데이터로 분류.
- 여기서는, 분류 성능 지표의 일관성을 위해 이상치를 positive로 분류하는 형태로 바꿈.

4 SVM 파이썬 실습

● One-Class SVM : 예측 함수

```
def predict_oneclass(X_test):  
    pred = []  
    for x in X_test:  
        val = decision_function_oneclass(x, alpha_oc, X_normal, K_oc, rho_oc, gamma=0.1)  
        if val >= 0:  
            pred.append(+1) # outlier  
        else:  
            pred.append(-1) # normal  
    return np.array(pred)
```

4 SVM 파이썬 실습

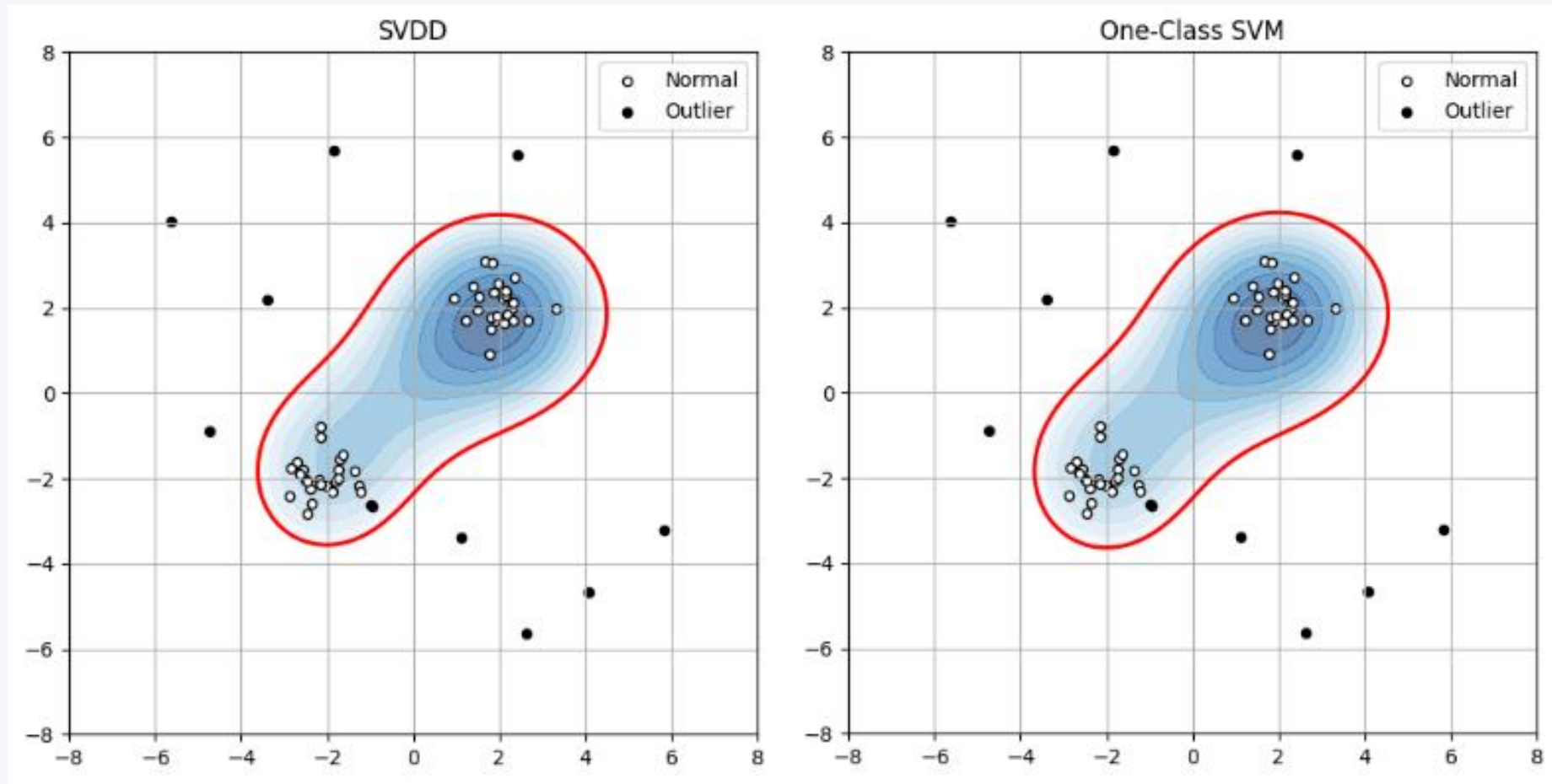
● SVDD 및 One-Class SVM 학습 : 하이퍼파라미터 튜닝

```
# Fit Support Vector Data Description (SVDD)
alpha_svdd, K_svdd, R_svdd = fit_svdd(X_train, C=1, gamma=0.3)
```

```
# Fit One-Class nu-SVM
alpha_oc, K_oc, rho_oc = fit_oneclass_svm(X_train, nu=0.2, gamma=0.3)
```

4 SVM 파이썬 실습

● 분류 결과 : 데이터 시각화



출처:파이썬 프로그램 실습내용 중

4 SVM 파이썬 실습

● 평가 지표 (Evaluation Metrics)

모델 예측 \ 실제 레이블	정상 (normal)	이상치 (outlier)
음성 (negative)	True Negative (TN)	False Negative (FN)
양성 (positive)	False Positive (FP)	True Positive (TP)

- One-Class Problem에서는 대부분의 데이터가 정상 (normal)
- 정확도(accuracy)를 사용하는 것이 적절할까?

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

4 SVM 파이썬 실습

● 평가 지표 (Evaluation Metrics)

- 이상치를 질병으로, 모델 예측을 진단검사 결과로 비유

- **민감도**: 실제 이상치들 중에서 모델이 이상치로 올바르게 예측한 비율

$$\text{Sensitivity} = \frac{TP}{TP + FN} \quad (\text{Positive Recall})$$

- **특이도**: 실제 정상 데이터들 중에서 모델이 정상으로 올바르게 예측한 비율

$$\text{Specificity} = \frac{TN}{TN + FP} \quad (\text{Negative Recall})$$

4 SVM 파이썬 실습

● 평가 지표 (Evaluation Metrics)

- 민감도/특이도 : 신의 관점에서 본 진단기기의 두 측면
- 양성/음성예측도 : 인간의 관점에서 본 진단기기의 두 측면

- 양성예측도: 모델이 이상치로 예측한 데이터 중 실제 이상치인 비율

$$\text{Positive Predictive Value (PPV)} = \frac{TP}{TP + FP} \quad (\text{Precision})$$

- 음성예측도: 모델이 정상으로 예측한 데이터 중 실제 정상 데이터인 비율

$$\text{Nevative Predictive Value (NPV)} = \frac{TN}{TN + FN}$$

4 SVM 파이썬 실습

● 분류 결과 : 평가 지표

```
TP = np.sum(y_pred_out_svdd == 1)
TN = np.sum(y_pred_normal_svdd == -1)
FP = np.sum(y_pred_normal_svdd == 1)
FN = np.sum(y_pred_out_svdd == -1)

svdd_sens = TP / (TP + FN)
svdd_spec = TN / (TN + FP)
svdd_PPV = TP / (TP + FP)
svdd_NPV = TN / (TN + FN)
print(f"[SVDD] Sens = {svdd_sens:.2f}, Spec = {svdd_spec:.2f}")
print(f"[SVDD] PPV = {svdd_PPV:.2f}, NPV = {svdd_NPV:.2f}")
```

```
[SVDD] Sens = 0.90, Spec = 1.00
```

```
[SVDD] PPV = 1.00, NPV = 0.98
```

★ 정리하기

- One-Class Problem

정상데이터 기반 이상치 판별 문제

- Support Vector Data Description

정상데이터를 감싸는 최소 크기의 초구를 찾는 방식

데이터의 중심에 초점을 맞추는 경향

하이퍼파라미터 C , γ 조절 (Soft-Margin, RBF)

- One-Class ν -SVM

원점을 기준으로 데이터를 분리하는 초평면을 찾는 방식

데이터의 경계에 초점을 맞추는 경향

하이퍼파라미터 ν , γ 조절 (RBF)

14 강

다음시간안내

최신 딥러닝

